

# INT-Klausur WS21/22 — Vollständige Lösung mit Erklärungen

Für Einsteiger ohne Vorkenntnisse: jede Aufgabe gelöst und verständlich erklärt

## Inhaltsverzeichnis

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>So liest du dieses Dokument</b>                                | <b>3</b>  |
| <b>Aufgabe 1 — Suche (9 P + 2 BP)</b>                             | <b>4</b>  |
| <b>Aufgabe 2 — Adversarial Search / Alpha-Beta (9 P)</b>          | <b>6</b>  |
| <b>Aufgabe 3 — CSP: Hütten-Rätsel (10 P)</b>                      | <b>8</b>  |
| <b>Aufgabe 4 — CSP: Arc-Konsistenz mit AC-1 (9 P)</b>             | <b>9</b>  |
| <b>Aufgabe 5 — CSP: Lösungsrelation (5 BP)</b>                    | <b>10</b> |
| <b>Aufgabe 6 — MDP: Value Iteration (6 P + 2 BP)</b>              | <b>11</b> |
| <b>Aufgabe 7 — Reinforcement Learning: Q-Learning (8 P)</b>       | <b>12</b> |
| <b>Aufgabe 8 — Markov-Kette (9 BP)</b>                            | <b>13</b> |
| <b>Aufgabe 9 — HMM (7 P)</b>                                      | <b>14</b> |
| <b>Aufgabe 10 — Full Joint Distribution (4 P + 1 BP)</b>          | <b>15</b> |
| <b>Aufgabe 11 — Bayes-Netz &amp; Variable Elimination (15 P)</b>  | <b>16</b> |
| <b>Aufgabe 12 — Bayesian Learning (7 P)</b>                       | <b>17</b> |
| <b>Aufgabe 13 — Decision Tree / Information Gain (6 P)</b>        | <b>18</b> |
| <b>Aufgabe 14 — Huffman-Entropie (5 BP)</b>                       | <b>19</b> |
| <b>Aufgabe 15 — Alldiff-Propagation / Marriage Theorem (4 BP)</b> | <b>20</b> |
| <b>Aufgabe 16 — Policy Iteration (7 BP)</b>                       | <b>21</b> |
| <b>Aufgabe 17 — Naive Bayes (5 P)</b>                             | <b>22</b> |
| <b>Aufgabe 18 — Expectimax (5 BP)</b>                             | <b>23</b> |



## So liest du dieses Dokument

Du hast den Kurs nie besucht — kein Problem. Jede Aufgabe ist in drei Blöcke geteilt:

- **Worum geht es?** — die Idee in einfachen Worten, ohne Fachjargon.
- **Lösung** — Schritt für Schritt, mit allen Rechenwegen (die will der Prüfer sehen).
- **Lernziel** — was du hier eigentlich lernen sollst und warum.

Rechenregel der Klausur: mit **4 signifikanten Stellen** rechnen und weitere Stellen **abschneiden** (nicht runden), z.B. aus 0,3456789 wird 0,3456.

## Aufgabe 1 — Suche (9 P + 2 BP)

### Worum geht es?

Stell dir vor, du suchst einen Weg durch ein Labyrinth. Der **Suchbaum** ist eine Landkarte aller möglichen Wege: oben die Wurzel (Start  $K_0$ ), darunter verzweigen sich die Möglichkeiten. Jeder Knoten hat einen Namen ( $K_i$ ) und die **aufsummierten Kosten** vom Start bis dorthin (die Zahl hinter dem „/“). Ein **Zielknoten** ist ein Ort, den wir erreichen wollen.

Die vier Suchverfahren unterscheiden sich nur darin, **in welcher Reihenfolge** sie die Knoten „aufmachen“ (besuchen):

- **Breitensuche (BFS)**: Ebene für Ebene, wie Wasser, das sich ausbreitet — erst alle Knoten in Tiefe 1, dann alle in Tiefe 2, ...
- **Tiefensuche (DFS)**: immer so tief wie möglich in den ersten Ast hinein, erst dann zurück.
- **Iterative Deepening (IDS)**: wiederholte Tiefensuche mit wachsendem Tiefenlimit 0, 1, 2, ... (jedes Mal von vorn).
- **Uniform Cost (UCS)**: immer den Knoten mit den **kleinsten Gesamtkosten** zuerst.

Der Baum dieser Aufgabe:

$$\begin{aligned} &K_0 \\ &\vdash K_1(9) : \vdash K_4(11) \rightarrow K_9(17); \vdash K_5(13) \rightarrow K_{10}(18), K_{11}(14) \\ &\vdash K_2(11) : \vdash K_6(12) \rightarrow K_{12}(16); \vdash K_7(15) \\ &\vdash K_3(7) : \vdash K_8(13) \rightarrow K_{13}(17) \end{aligned}$$

Ebenen: Tiefe 0 =  $K_0$ ; Tiefe 1 =  $K_1, K_2, K_3$ ; Tiefe 2 =  $K_4, K_5, K_6, K_7, K_8$ ; Tiefe 3 =  $K_9, K_{10}, K_{11}, K_{12}, K_{13}$ .

### Lösung

**(i) Breitensuche, Ziele  $K_7$  und  $K_{10}$ .** Ebene für Ebene, Kinder von links nach rechts:

$$K_0, K_1, K_2, K_3, K_4, K_5, K_6, \mathbf{K_7}.$$

$K_7$  liegt in Tiefe 2 und wird dort vor  $K_{10}$  (Tiefe 3) gefunden. **Stopp bei  $K_7$ .**

**(ii) Tiefensuche, Ziele  $K_8$  und  $K_{10}$ .** Immer zuerst ganz nach links in die Tiefe:

$$K_0, K_1, K_4, K_9, K_5, \mathbf{K_{10}}.$$

Von  $K_0$  nach  $K_1$ , dann  $K_4$ , dann  $K_9$  (Sackgasse, zurück), dann  $K_5$ , dann dessen erstes Kind  $K_{10}$  — ein Ziel. **Stopp bei  $K_{10}$**  ( $K_8$  läge erst im rechten Ast).

**(iii) Iterative Deepening, Ziele  $K_7$  und  $K_{11}$ .** Tiefensuche mit Limit 0, 1, 2, jeweils neu von der Wurzel:

$$\begin{aligned} L = 0 : & K_0 \\ L = 1 : & K_0, K_1, K_2, K_3 \\ L = 2 : & K_0, K_1, K_4, K_5, K_2, K_6, \mathbf{K_7} \end{aligned}$$

Bei  $L = 2$  wird  $K_7$  (Tiefe 2) erreicht, bevor  $K_{11}$  (Tiefe 3) dran wäre. **Stopp bei  $K_7$ .**

**(iv) Uniform Cost, Ziele  $K_7$  und  $K_9$ .** Immer kleinste Gesamtkosten zuerst. Reihenfolge der Expansion (bei Kostengleichheit ist die Reihenfolge frei, daher „könnte“):

$K_0(0)$ ,  $K_3(7)$ ,  $K_1(9)$ ,  $K_2(11)$ ,  $K_4(11)$ ,  $K_6(12)$ ,  $K_5(13)$ ,  $K_8(13)$ ,  $K_{11}(14)$ ,  **$K_7(15)$ .**

$K_7$  hat Kosten 15, das Ziel  $K_9$  hat 17. Da UCS erst beim **Herausnehmen** (Expandieren) auf Ziel prüft, wird alles mit Kosten  $< 15$  vorher expandiert.  **$K_7$  zuerst gefunden (Kosten 15).**

**(v) WAHR.** Tiefensuche hängt von der Tiefe ab, nicht vom Verzweigungsfaktor. Liegt das Optimum zufällig auf dem zuerst verfolgten Pfad, findet DFS es auch bei unendlichem  $b$  — *garantiert* ist das nicht, aber möglich („kann“).

**(vi) FALSCH.** Gegenbeispiel: das Ziel liegt flach im **rechten** Ast, aber der **linke** Ast ist sehr tief. DFS taucht zuerst tief in den linken Ast ( $O(b \cdot m)$  Speicher), bevor es das flache Ziel rechts findet — und braucht dann mehr Speicher als BFS, das das flache Ziel sofort findet. Also ist BFS **nicht immer** teurer.

**(vii) WAHR.** Bei gleichen Schrittkosten ist die **flachste** Lösung automatisch die billigste. IDS findet die flachste zuerst → also die optimale.

**(viii) Bonus:** Jeder Schritt kostet mindestens  $\alpha$ . Bei Gesamtkosten  $C^*$  kann der Pfad höchstens

$$\left\lfloor \frac{C^*}{\alpha} \right\rfloor \text{ Schritte tief sein,}$$

denn mehr Schritte  $\geq \alpha$  würden die Kosten  $C^*$  übersteigen.

(i)  $K_0, K_1, K_2, K_3, K_4, K_5, K_6, K_7$     (ii)  $K_0, K_1, K_4, K_9, K_5, K_{10}$   
 (iii)  $K_0 \mid K_0 K_1 K_2 K_3 \mid K_0 K_1 K_4 K_5 K_2 K_6 K_7$     (iv)  $K_0, K_3, K_1, K_2, K_4, K_6, K_5, K_8, K_{11}, K_7$   
 (v) WAHR    (vi) FALSCH    (vii) WAHR    (viii)  $\lfloor C^*/\alpha \rfloor$

## Lernziel

Du sollst die vier **uninformierten** Suchstrategien mechanisch durchspielen können und ihre Eigenschaften verstehen: BFS/IDS finden bei gleichen Kosten die kürzeste Lösung, UCS findet immer die billigste, DFS ist speichersparsam, aber weder vollständig noch optimal. Der Bonusteil trainiert das Denken in Schranken (Tiefe  $\leftrightarrow$  Kosten).

## Aufgabe 2 — Adversarial Search / Alpha-Beta (9 P)

### Worum geht es?

Zwei Spieler wechseln sich ab. **MAX** (Dreieck mit Spitze nach oben) will den Wert **maximieren**, **MIN** (Spitze nach unten) will ihn **minimieren**. Ganz unten stehen feste Zahlen (die Blätter). Der Wert wandert von unten nach oben: an MAX-Knoten nimmt man das **Maximum** der Kinder, an MIN-Knoten das **Minimum**.

**Alpha-Beta** ist ein Trick, um sich Arbeit zu sparen: Wenn schon feststeht, dass ein Teilbaum das Ergebnis nicht mehr ändern kann, „schneidet“ man ihn ab (Pruning). Dafür trägt man zwei Zahlen mit:

- $\alpha$  = das Beste, was MAX bisher sicher hat (untere Schranke),
- $\beta$  = das Beste, was MIN bisher sicher hat (obere Schranke).

**Abschneiden, sobald**  $\alpha \geq \beta$ . Start:  $\alpha = -\infty$ ,  $\beta = +\infty$ .

Der Baum (von links nach rechts): Wurzel = MAX, darunter zwei MIN-Knoten.

#### Wurzel (MAX)

└ **L (MIN)** : Blatt 5,  $\text{MAX}_a(7, 9)$

└ **R (MIN)** :  $\text{MAX}_b(8, \text{MIN}_d(7, 9, 4))$ ,  $\text{MAX}_c(9, 6)$ , Blatt 4, Blatt 3

### Lösung (Durchlauf von links nach rechts)

#### Linker Teilbaum L (MIN):

- Blatt 5 → vorläufig  $v = 5$ , also  $\beta = 5$  für L.
- Aufruf  $\text{MAX}_a$  mit  $\alpha = -\infty$ ,  $\boxed{\beta = 5}$ . Erstes Blatt 7:  $v = 7 \geq \beta = 5 \rightarrow \beta\text{-Cut}$ , Blatt 9 wird **abgeschnitten**.  $\text{MAX}_a = 7$ .
- $L = \min(5, 7) = 5$ .

Zurück zur Wurzel:  $v = 5$ , also  $\boxed{\alpha = 5}$  für den Aufruf des rechten Teilbaums.

#### Rechter Teilbaum R (MIN) mit $\alpha = 5$ , $\beta = +\infty$ :

- Aufruf  $\text{MAX}_b$  mit  $\alpha = 5$ ,  $\boxed{\beta = +\infty}$  (R hat noch keinen Wert). Blatt 8:  $v = 8$ ,  $\alpha = 8$ . Aufruf  $\text{MIN}_d$  mit  $\boxed{\alpha = 8}$ ,  $\beta = +\infty$ . Erstes Blatt 7:  $v = 7 \leq \alpha = 8 \rightarrow \alpha\text{-Cut}$ , Blätter 9 und 4 **abgeschnitten**.  $\text{MIN}_d = 7$ . Also  $\text{MAX}_b = \max(8, 7) = 8$ .
- R aktualisiert:  $v = 8$ ,  $\beta = 8$ .
- Aufruf  $\text{MAX}_c$  mit  $\alpha = 5$ ,  $\boxed{\beta = 8}$ . Blatt 9:  $v = 9 \geq \beta = 8 \rightarrow \beta\text{-Cut}$ , Blatt 6 **abgeschnitten**.  $\text{MAX}_c = 9$ .
- R:  $v = \min(8, 9) = 8$ . Nächstes Kind Blatt 4:  $v = \min(8, 4) = 4 \leq \alpha = 5 \rightarrow \alpha\text{-Cut}$ , Blatt 3 **abgeschnitten**.  $R = 4$ .

Wurzel =  $\max(L=5, R=4) = 5$ .

**(b) Optimale Besuchsreihenfolge (minimale Blätterzahl).** Wähle in jedem Knoten die beste Kinderreihenfolge:

1. In L zuerst Blatt 5  $\rightarrow \beta = 5$ ; dann in  $\text{MAX}_a$  ein Blatt  $\geq 5$  (z.B. 7)  $\rightarrow$  sofort Cut.  $L = 5$ ,  $\alpha = 5$ . (2 Blätter)
2. In R zuerst direkt das kleine Blatt 3:  $3 \leq \alpha = 5 \rightarrow$  ganz R abgeschnitten. (1 Blatt)

Besucht: 5 (1.), 7 (2.), 3 (3.)  $\Rightarrow$  **nur 3 Blätter**, alle übrigen „-“.

Knotenwerte:  $\text{MAX}_a = 7$ ,  $L = 5$ ,  $\text{MIN}_d = 7$ ,  $\text{MAX}_b = 8$ ,  $\text{MAX}_c = 9$ ,  $R = 4$ , **Wurzel** = 5.  
 $\alpha, \beta$ -Stellen: Wurzel  $\rightarrow$  R:  $\alpha = 5$ ; L  $\rightarrow$   $\text{MAX}_a$ :  $\beta = 5$ ; R  $\rightarrow$   $\text{MAX}_b$ :  $\beta = \infty$ ;  $\text{MAX}_b \rightarrow$   $\text{MIN}_d$ :  $\alpha = 8$ ; R  $\rightarrow$   $\text{MAX}_c$ :  $\beta = 8$ .  
 Abgeschnitten: 9 (bei  $\text{MAX}_a$ ), 9 und 4 (bei  $\text{MIN}_d$ ), 6 (bei  $\text{MAX}_c$ ), 3 (bei R).  
 (b) Optimale Reihenfolge besucht nur 3 Blätter: 5, 7, 3.

## Lernziel

Du sollst Minimax verstehen (abwechselndes Max/Min) und Alpha-Beta **sorgfältig** ausführen: die Schranken  $\alpha, \beta$  korrekt mitführen und genau erkennen, **wann** ein Ast wegfallen darf. Teil (b) zeigt: bei idealer Zugreihenfolge halbiert sich der Suchaufwand — der Kern, warum Alpha-Beta so wichtig ist.

## Aufgabe 3 — CSP: Hütten-Rätsel (10 P)

### Worum geht es?

Ein **Constraint-Satisfaction-Problem** (CSP) ist ein Rätsel mit **Variablen**, die aus einer festen Menge an **Werten** (Domain) belegt werden, wobei **Bedingungen (Constraints)** erfüllt sein müssen. Hier: 3 Hütten (Positionen 1, 2, 3), jede mit einer Nationalität, einem Haustier, einem Getränk.

### Lösung

**(i) Modellierung.** Variablen (ihr Wert ist jeweils die Hüttennummer  $\in \{1, 2, 3\}$ ):

Schweizer, Franzose, Belgier; Hund, Häschen, Hamster; Sekt, Bier, Tee.

Constraints aus den Fakten:

- je Kategorie **alldiff** (Nationalitäten, Tiere, Getränke paarweise verschieden),
- $\text{Pos}(\text{Schweizer}) = \text{Pos}(\text{Hund})$  (iii),
- $|\text{Pos}(\text{Franzose}) - \text{Pos}(\text{Bier})| = 1$  (iv),
- $\text{Pos}(\text{Belgier}) = \text{Pos}(\text{Bier})$  (v),
- $\text{Pos}(\text{Sekt}) = 1$  (vi),
- $\text{Pos}(\text{Franzose}) = 3$  (vii),
- $\text{Pos}(\text{Tee}) = \text{Pos}(\text{Häschen})$  (viii).

**(ii) Lösen (strukturiert).**

1. Franzose = 3 (vii). Sekt = 1 (vi).
2. Franzose (Hütte 3) wohnt neben dem Biertrinker (iv); Nachbar von Hütte 3 ist nur Hütte 2  $\rightarrow$  Bier = 2.
3. Belgier trinkt Bier (v)  $\rightarrow$  Belgier = 2. Übrig bleibt Schweizer = 1.
4. Getränke: Hütte 1 = Sekt, Hütte 2 = Bier, also Hütte 3 = **Tee**. Damit trinkt der **Franzose Tee**.
5. Teetrinker besitzt Häschen (viii)  $\rightarrow$  Häschen = 3 (Franzose).
6. Schweizer besitzt Hund (iii)  $\rightarrow$  Hund = 1. Übrig: Hamster = 2 (Belgier).

**Wer trinkt Tee?** Der Franzose (Hütte 3). **Wem gehört der Hamster?** Dem Belgier (Hütte 2).

Hütte 1: Schweizer, Hund, Sekt | Hütte 2: Belgier, Hamster, Bier | Hütte 3: Franzose, Häschen, Tee.

### Lernziel

Du übst, eine Textbeschreibung sauber in ein formales CSP zu übersetzen (Variablen + Domains + Constraints) und es dann durch **logisches Schließen statt Raten** zu lösen. Zentrales Werkzeug: feste Fakten zuerst nutzen, dann Kette von Folgerungen.

## Aufgabe 4 — CSP: Arc-Konsistenz mit AC-1 (9 P)

### Worum geht es?

Ein Bogen  $x_i \rightarrow x_j$  ist **arc-konsistent**, wenn es zu **jedem** erlaubten Wert von  $x_i$  **mindestens einen** passenden Wert von  $x_j$  gibt, der das Constraint erfüllt. Gibt es zu einem Wert keinen Partner, wird er aus der Domain gestrichen. **AC-1** prüft in Runden **alle** Bögen (jedes Constraint in **beide** Richtungen), bis eine ganze Runde ohne Änderung bleibt.

Gegeben:  $D_1 = \{2, 3, 4, 5\}$ ,  $D_2 = \{2, 3, 4\}$ ,  $D_3 = \{3, 4, 5\}$ ,  $D_4 = \{1, 2, 3, 4, 5\}$ ,  $D_5 = \{1, 2, 3, 4\}$  mit  $C_{12}: x_1 < x_2$ ,  $C_{24}: x_2 > x_4$ ,  $C_{34}: x_3 \leq x_4$ ,  $C_{35}: x_3 > x_5$ ,  $C_{51}: x_5 < x_1$ .

### Lösung

**(i) Constraintgraph:** Knoten  $x_1, \dots, x_5$ ; Kanten  $x_1 - x_2$ ,  $x_2 - x_4$ ,  $x_3 - x_4$ ,  $x_3 - x_5$ ,  $x_5 - x_1$  (Ring  $x_1 - x_2 - x_4 - x_3 - x_5 - x_1$ ).

**(ii) AC-1 (nur die Streichungen):**

- $x_1 < x_2$ : Aus  $D_1$  fällt 4,5 (kein  $x_2 > 4$  da  $\max D_2 = 4$ )  $\rightarrow D_1 = \{2, 3\}$ . Rückrichtung  $x_2 > x_1$ : aus  $D_2$  fällt 2  $\rightarrow D_2 = \{3, 4\}$ .
- $x_2 > x_4$ : passt noch. Rückrichtung  $x_4 < x_2$ : aus  $D_4$  fällt 4,5  $\rightarrow D_4 = \{1, 2, 3\}$ .
- $x_3 \leq x_4$ : aus  $D_3$  fällt 4,5 (kein  $x_4 \geq 4$ )  $\rightarrow D_3 = \{3\}$ . Rückrichtung  $x_4 \geq x_3$ : aus  $D_4$  fällt 1,2  $\rightarrow D_4 = \{3\}$ .
- $x_3 > x_5$ : passt. Rückrichtung  $x_5 < x_3$ : aus  $D_5$  fällt 3,4  $\rightarrow D_5 = \{1, 2\}$ .

**Nächste Runde** (Domains haben sich geändert, alles neu prüfen):

- $x_2 > x_4$  mit  $D_4 = \{3\}$ : aus  $D_2$  fällt 3 ( $3 > 3$  falsch)  $\rightarrow D_2 = \{4\}$ .
- Alle weiteren Prüfungen ändern nichts mehr  $\rightarrow$  **Stopp**.

Arc-konsistentes Netz:  $D_1 = \{2, 3\}$ ,  $D_2 = \{4\}$ ,  $D_3 = \{3\}$ ,  $D_4 = \{3\}$ ,  $D_5 = \{1, 2\}$ .  
(Probe: z.B.  $(x_1, x_2, x_3, x_4, x_5) = (2, 4, 3, 3, 1)$  erfüllt alle Constraints.)

### Lernziel

Du lernst **Constraint-Propagation**: ohne zu raten, Werte entfernen, die nie in einer Lösung vorkommen können. AC-1 macht das systematisch rundenweise. Wichtig: jedes Constraint erzeugt **zwei** Bögen, und  $\leq$  ist nicht dasselbe wie  $<$ .

## Aufgabe 5 — CSP: Lösungsrelation (5 BP)

### Worum geht es?

Die **Lösungsrelation** ist die Liste **aller** vollständigen Belegungen  $(x_1, x_2, x_3, x_4)$ , die **gleichzeitig alle** Constraint-Tabellen erfüllen. Man baut sie durch schrittweises **Verbinden (Join)** der Tabellen: nur Tupel kombinieren, die auf gemeinsamen Variablen denselben Wert haben.

### Lösung

Wir gehen die vier Kandidaten aus  $x_1x_2$  durch und prüfen Verträglichkeit mit den übrigen Tabellen:

- $(x_1, x_2) = (3, 1)$ :  $x_1x_3$  erzwingt  $x_3 = 7$ , aber  $x_2x_3$  mit  $x_2=1$  erzwingt  $x_3 = 5 \rightarrow$  **Widerspruch**, entfällt.
- $(4, 2)$ :  $x_1x_3 \rightarrow x_3 = 7$ ;  $x_2x_3$  mit  $x_2=2 \rightarrow x_3 = 5 \rightarrow$  **Widerspruch**, entfällt.
- $(5, 1)$ :  $x_1x_3 \Rightarrow x_3 = 5$ ;  $x_2x_3$  mit  $x_2=1 \Rightarrow x_3 = 5$  (ok).  $x_1x_4$  erlaubt  $x_4 \in \{2, 3\}$ ,  $x_2x_4$  mit  $x_2=1 \Rightarrow x_4 = 2$ .  $x_3x_4 = (5, 2)$  (ok)  $\rightarrow$  **Lösung**  $(5, 1, 5, 2)$ .
- $(5, 3)$ :  $x_1x_3 \Rightarrow x_3 = 5$ ;  $x_2x_3$  mit  $x_2=3 \Rightarrow x_3 = 5$  (ok).  $x_2x_4$  mit  $x_2=3 \Rightarrow x_4 = 3$ ;  $x_1x_4 = (5, 3)$  (ok);  $x_3x_4 = (5, 3)$  (ok)  $\rightarrow$  **Lösung**  $(5, 3, 5, 3)$ .

|                                                                               |
|-------------------------------------------------------------------------------|
| Lösungsrelation = $\{ (x_1, x_2, x_3, x_4) = (5, 1, 5, 2), (5, 3, 5, 3) \}$ . |
|-------------------------------------------------------------------------------|

### Lernziel

Du verstehst, dass ein CSP eine **Menge von Lösungen** definiert, und wie man sie durch Join/Verträglichkeitsprüfung konstruiert. Trick: immer das nächste Constraint wählen, das eine bereits belegte Variable teilt — so bleiben die Zwischentabellen klein.

## Aufgabe 6 — MDP: Value Iteration (6 P + 2 BP)

### Worum geht es?

Ein **Markov-Entscheidungsprozess** (MDP) modelliert Entscheiden unter Zufall: In einem **Zustand**  $s$  wählt man eine **Aktion**  $a$ , landet mit Wahrscheinlichkeit  $T(s, a, s')$  in  $s'$  und kassiert eine **Belohnung**  $R$ . Der **Discount**  $\gamma$  gewichtet Zukünftiges schwächer. Der **Wert**  $V(s)$  ist der beste erwartete Gesamtertrag ab  $s$ .

**Q-Wert** (Ertrag einer konkreten Aktion) und **Bellman-Update**:

$$Q(s, a) = \sum_{s'} T(s, a, s') [R + \gamma V(s')], \quad V_{k+1}(s) = \max_a Q(s, a).$$

Value Iteration startet mit  $V_0 = 0$  und verbessert schrittweise. Hier  $\gamma = 0,4$ .

### Lösung

$$V_0(\text{Cool}) = V_0(\text{Warm}) = 0.$$

$V_1$ :

$$Q(\text{Cool}, \text{Slow}) = 1 \cdot (1 + 0,4 \cdot 0) = 1, \quad Q(\text{Cool}, \text{Fast}) = 0,4(2) + 0,6(2) = 2 \Rightarrow V_1(\text{Cool}) = 2.$$

$$Q(\text{Warm}, \text{Slow}) = 0,2(1) + 0,8(1) = 1, \quad Q(\text{Warm}, \text{Fast}) = -6 \Rightarrow V_1(\text{Warm}) = 1.$$

$V_2$  (jetzt mit  $V_1$  in der Klammer):

$$Q(\text{Cool}, \text{Slow}) = 1 + 0,4 \cdot 2 = 1,8, \quad Q(\text{Cool}, \text{Fast}) = 0,4(2 + 0,4 \cdot 2) + 0,6(2 + 0,4 \cdot 1) = 1,12 + 1,44 = 2,56. \\ \Rightarrow V_2(\text{Cool}) = 2,56.$$

$$Q(\text{Warm}, \text{Slow}) = 0,2(1 + 0,4 \cdot 2) + 0,8(1 + 0,4 \cdot 1) = 0,36 + 1,12 = 1,48, \quad Q(\text{Warm}, \text{Fast}) = -6 + 0,4 \cdot 1 = -5,6. \\ \Rightarrow V_2(\text{Warm}) = 1,48.$$

**(b) Policy Extraction** ( $\gamma = 0,6$ ,  $V^*(\text{Cool}) = 3,977$ ,  $V^*(\text{Warm}) = 2,841$ ): Für jeden Zustand die Aktion mit dem größten Q-Wert wählen.

$$\text{Cool: } Q_{\text{Slow}} = 1 + 0,6 \cdot 3,977 = 3,386, \quad Q_{\text{Fast}} = 0,4(2 + 0,6 \cdot 3,977) + 0,6(2 + 0,6 \cdot 2,841) = 3,977. \\ \Rightarrow \pi^*(\text{Cool}) = \text{Fast}.$$

$$\text{Warm: } Q_{\text{Slow}} = 0,2(2 + 0,6 \cdot 3,977) + 0,8(2 + 0,6 \cdot 2,841) = 2,841, \quad Q_{\text{Fast}} = -6 + 0,6 \cdot 2,841 = -4,295. \\ \Rightarrow \pi^*(\text{Warm}) = \text{Slow}.$$

$$V_0 = \langle 0, 0 \rangle; \quad V_1(\text{Cool}) = 2, \quad V_1(\text{Warm}) = 1; \quad V_2(\text{Cool}) = 2,56, \quad V_2(\text{Warm}) = 1,48. \\ \pi^*(\text{Cool}) = \text{Fast}, \quad \pi^*(\text{Warm}) = \text{Slow}.$$

### Lernziel

Du lernst das **Bellman-Update** als Herz der dynamischen Programmierung: Werte wiederholt aus den Werten der Folgezustände berechnen. „Policy Extraction“ zeigt, wie aus Werten die beste Handlung abgeleitet wird — die Verbindung von *Bewerten* zu *Entscheiden*.

## Aufgabe 7 — Reinforcement Learning: Q-Learning (8 P)

### Worum geht es?

Beim **Q-Learning** kennt man das MDP **nicht** — man lernt nur aus **Erfahrungen** (Samples) der Form  $(s, a, s', r)$ : „war in  $s$ , tat  $a$ , landete in  $s'$ , bekam  $r$ “. Man aktualisiert eine Tabelle von Q-Werten:

$$Q(s, a) \leftarrow (1 - \alpha) Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a')].$$

Der eckige Klammerausdruck ist das **Lernziel** aus diesem Sample;  $\alpha$  bestimmt, wie stark wir es übernehmen. Hier  $\gamma = 0,5$ ,  $\alpha = 0,5$ , Start alle  $Q = 0$ . (Aktion „Go“ = „Run“.)

### Lösung (Sample für Sample)

1.  $(B, \text{Stop}, B, 1)$ :  $\max Q(B, \cdot) = 0$ .  $Q(B, \text{Stop}) = 0,5 \cdot 0 + 0,5(1 + 0,5 \cdot 0) = \frac{1}{2}$ .
2.  $(A, \text{Run}, B, \frac{6}{5})$ :  $\max Q(B, \cdot) = \frac{1}{2}$ .  $Q(A, \text{Run}) = 0,5(\frac{6}{5} + 0,5 \cdot \frac{1}{2}) = 0,725$ .
3.  $(A, \text{Stop}, A, 2)$ :  $\max Q(A, \cdot) = 0,725$ .  $Q(A, \text{Stop}) = 0,5(2 + 0,5 \cdot 0,725) = 1,18125$ .
4.  $(B, \text{Run}, B, -1)$ :  $\max Q(B, \cdot) = \frac{1}{2}$ .  $Q(B, \text{Run}) = 0,5(-1 + 0,5 \cdot \frac{1}{2}) = -0,375$ .
5.  $(B, \text{Stop}, A, -\frac{3}{2})$ :  $\max Q(A, \cdot) = 1,18125$ .  $Q(B, \text{Stop}) = 0,5 \cdot \frac{1}{2} + 0,5(-\frac{3}{2} + 0,5 \cdot 1,18125) = -0,20469$ .

Endtabelle:

|   | Run    | Stop     |
|---|--------|----------|
| A | 0,725  | 1,18125  |
| B | -0,375 | -0,20469 |

**(b) Policy:** je Zustand die Aktion mit größerem Q-Wert.

$Q(A, \text{Run})=0,725$ ,  $Q(A, \text{Stop})=1,18125$ ,  $Q(B, \text{Run})= - 0,375$ ,  $Q(B, \text{Stop})= - 0,20469$ .  
 $\pi^*(A) = \text{Stop}$ ,  $\pi^*(B) = \text{Stop}$  ( $-0,20469 > -0,375$ ).

### Lernziel

Du verstehst den Unterschied zu Aufgabe 6: hier ist das Modell **unbekannt**, man lernt aus Stichproben. Die Update-Regel mischt alten Wert und neue Beobachtung. Der  $\max_{a'}$ -Term im Ziel macht Q-Learning **off-policy** (es lernt den optimalen Wert, egal welche Aktion tatsächlich gewählt wurde).

## Aufgabe 8 — Markov-Kette (9 BP)

### Worum geht es?

Eine **Markov-Kette** beschreibt Zustandswechsel durch Wahrscheinlichkeiten: Wen du als Nächstes triffst, hängt nur davon ab, wen du **gerade** getroffen hast. Die **Transitionsmatrix**  $T$  enthält in Zeile  $i$  die Wahrscheinlichkeiten, von  $i$  zu jedem nächsten Zustand zu wechseln (jede Zeile summiert zu 1). Zustände: Ritter  $r$ , Schurke  $s$ , Kobold  $k$ .

### Lösung

(a) „Restliche %“ auffüllen (Zeilensumme 1). Verhältnis  $r : s : k = 1 : 2 : 1 \rightarrow$  normiert.

$$P_0 = \langle r:\frac{1}{4}, s:\frac{1}{2}, k:\frac{1}{4} \rangle, \quad T = \begin{array}{c|ccc} & r & s & k \\ \hline r & 0,55 & 0,20 & 0,25 \\ s & 0,30 & 0,40 & 0,30 \\ k & 0,40 & 0,35 & 0,25 \end{array}$$

(b) **Ein Schritt**  $P_1(j) = \sum_i P_0(i) T_{ij}$ :

$$P_1(r) = 0,25 \cdot 0,55 + 0,5 \cdot 0,30 + 0,25 \cdot 0,40 = 0,3875,$$

$$P_1(s) = 0,25 \cdot 0,20 + 0,5 \cdot 0,40 + 0,25 \cdot 0,35 = 0,3375, \quad P_1(k) = 0,275.$$

(c) **Stationäre Verteilung**  $P = P \cdot T$  mit  $P(r) + P(s) + P(k) = 1$ :

$$0,45 P(r) = 0,30 P(s) + 0,40 P(k), \quad 0,60 P(s) = 0,20 P(r) + 0,35 P(k), \quad P(r) + P(s) + P(k) = 1.$$

Lösung:

$$P = \langle r:0,4353, s:0,2997, k:0,2650 \rangle.$$

(d) Bei stationärer Verteilung, 3 unabhängige Begegnungen, kein Kobold:

$$(1 - P(k))^3 = (1 - 0,2650)^3 = 0,7350^3 = 0,3971.$$

$$P_0 = \langle 0,25; 0,5; 0,25 \rangle; \quad P_1 = \langle 0,3875; 0,3375; 0,275 \rangle;$$
$$P = \langle r:0,4353; s:0,2997; k:0,2650 \rangle; \quad P(\text{kein Kobold unter 3}) = 0,3971.$$

### Lernziel

Du lernst, eine Textbeschreibung in eine Transitionsmatrix zu übersetzen, eine Verteilung einen Schritt vorwärts zu propagieren und die **stationäre Verteilung** (langfristiges Gleichgewicht) über ein Gleichungssystem zu bestimmen. Teil (d) verknüpft das mit i.i.d.-Wahrscheinlichkeiten.

## Aufgabe 9 — HMM (7 P)

### Worum geht es?

Ein **Hidden Markov Model** hat **versteckte** Zustände  $W_t$  (die wir nicht sehen) und **Beobachtungen**  $O_t$  (die wir sehen). Wir wollen aus den Beobachtungen auf den versteckten Zustand schließen. Werkzeug: der **Forward-Algorithmus** mit zwei abwechselnden Schritten:

- **Vorhersage** (predict): einen Zeitschritt weiter,  $P(W_{t+1}) = \sum_{w_t} P(W_{t+1}|w_t)P(w_t| \dots)$ .
- **Korrektur** (update): mit der neuen Beobachtung gewichten,  $P(W|o) \propto P(o|W) P(W)$ , dann **normieren**.

Beobachtungen:  $O_1 = +o$ ,  $O_2 = -o$ .

### Lösung

**Schritt 1 —  $W_1$  mit  $O_1 = +o$  korrigieren:**  $P(w_1|+o) \propto P(+o|w_1)P(w_1)$ .

$$+w: 0,7 \cdot 0,3 = 0,21, \quad -w: 0,9 \cdot 0,7 = 0,63 \Rightarrow \text{normiert } \langle +w: 0,25, -w: 0,75 \rangle.$$

**Schritt 2 — Vorhersage auf  $W_2$ :**

$$P(+w) = 0,6 \cdot 0,25 + 0,2 \cdot 0,75 = 0,30, \quad P(-w) = 0,4 \cdot 0,25 + 0,8 \cdot 0,75 = 0,70.$$

**Schritt 3 —  $W_2$  mit  $O_2 = -o$  korrigieren:**  $P(w_2| \dots) \propto P(-o|w_2) P(w_2)$ .

$$+w: 0,3 \cdot 0,30 = 0,09, \quad -w: 0,1 \cdot 0,70 = 0,07 \Rightarrow \text{Summe } 0,16.$$

$$\text{Normiert: } \langle +w: \frac{0,09}{0,16}, -w: \frac{0,07}{0,16} \rangle.$$

$$P(W_2 | O_1 = +o, O_2 = -o) = \langle +w: 0,5625, -w: 0,4375 \rangle.$$

### Lernziel

Du verstehst das Wechselspiel **Vorhersage**  $\leftrightarrow$  **Korrektur** (Filtering). Kernideen: Übergangsmodell bewegt die Verteilung in der Zeit, Beobachtung schärft sie, und nach jeder Beobachtung wird **normiert**, damit die Wahrscheinlichkeiten wieder 1 ergeben.

## Aufgabe 10 — Full Joint Distribution (4 P + 1 BP)

### Worum geht es?

Die **gemeinsame Verteilung**  $P(A, B, C)$  listet für **jede** Kombination die Wahrscheinlichkeit (Summe aller Zeilen = 1). Daraus bekommt man alles durch:

- **Marginalisieren** (Aufsummieren nicht gefragter Variablen),
- **Bedingen**  $P(X|Y) = \frac{P(X,Y)}{P(Y)}$  (nur passende Zeilen, dann durch deren Summe teilen).

### Lösung

(a)  $P(+a) = 0,3+0,05+0,05+0,15 = 0,55$ ,  $P(-a) = 0,45$ .  $P(A) = \langle 0,55; 0,45 \rangle$ .

(b) Nur  $-c$ -Zeilen;  $P(-c) = 0,05+0,15+0,1+0,05 = 0,35$ . Teilen durch 0,35:

$$P(A, B | -c) = \langle +a+b:0,1428, +a-b:0,4285, -a+b:0,2857, -a-b:0,1428 \rangle.$$

(c)  $P(+a, +b) = 0,35$ ,  $P(+a, -b) = 0,20$ ,  $P(+a) = 0,55$ :

$$P(B | +a) = \langle +b:0,6363, -b:0,3636 \rangle.$$

(d)  $P(+a | +b, -c) = \frac{P(+a, +b, -c)}{P(+b, -c)} = \frac{0,05}{0,05 + 0,1} = \frac{0,05}{0,15} = 0,3333$ .

(e)  $A$  ist unabhängig von  $B$  genau dann, wenn für **alle** Werte gilt

$$P(A, B) = P(A) \cdot P(B).$$

(Probe:  $P(+a, +b) = 0,35$  vs.  $P(+a)P(+b) = 0,55 \cdot 0,65 = 0,3575$  — nicht gleich, also **nicht** unabhängig.)

(a)  $\langle 0,55; 0,45 \rangle$  (b)  $\langle 0,1428; 0,4285; 0,2857; 0,1428 \rangle$  (c)  $\langle 0,6363; 0,3636 \rangle$  (d) 0,3333  
(e) prüfe  $P(A, B) = P(A) P(B)$  für alle Werte.

### Lernziel

Du übst die drei Grundoperationen der Wahrscheinlichkeit an einer Tabelle: **marginalisieren**, **bedingen**, **normieren** — und die formale Definition von **Unabhängigkeit**. Das ist die Basis für alles, was mit Bayes-Netzen kommt.

## Aufgabe 11 — Bayes-Netz & Variable Elimination (15 P)

### Worum geht es?

Ein **Bayes-Netz** ist eine kompakte Darstellung einer gemeinsamen Verteilung durch einen Graphen: jeder Knoten hängt (über eine CPT, bedingte Wahrscheinlichkeitstabelle) nur von seinen **Eltern** ab. Hier: Raub  $R$  und Erdbeben  $E$  verursachen Alarm  $A$ ; der Alarm führt zu Anrufen von John  $J$  und Mary  $M$ .

$$P(R, E, A, J, M) = P(R)P(E)P(A|R, E)P(J|A)P(M|A).$$

### Lösung

(i) Eine **vollständige** Belegung = einfach die passenden CPT-Werte multiplizieren:

$$\begin{aligned} P(\neg a, \neg j, m, \neg r, \neg e) &= P(\neg r)P(\neg e)P(\neg a|\neg r, \neg e)P(\neg j|\neg a)P(m|\neg a) \\ &= 0,95 \cdot 0,9 \cdot 0,95 \cdot 0,90 \cdot 0,05 = 0,03655. \end{aligned}$$

(ii) **Variable Elimination** für  $P(E \mid +j, -m)$ . Versteckte Variablen:  $R, A$ . Evidenz  $J=+$ ,  $M=-m$  einsetzen.

*Faktor über  $A$  zusammenfassen* — definiere  $g(R, E) = \sum_A P(A|R, E) P(+j|A) P(-m|A)$ . Die von  $A$  abhängigen Evidenzteile:

$$A=T: 0,95 \cdot 0,40 = 0,38, \quad A=F: 0,10 \cdot 0,95 = 0,095.$$

Also  $g(R, E) = P(a|R, E) \cdot 0,38 + P(\neg a|R, E) \cdot 0,095$ :

$$g(T, T) = 0,36575, \quad g(T, F) = 0,3515, \quad g(F, T) = 0,19475, \quad g(F, F) = 0,10925.$$

*Variable  $R$  eliminieren* —  $h(E) = \sum_R P(R) g(R, E)$ :

$$h(e) = 0,05 \cdot 0,36575 + 0,95 \cdot 0,19475 = 0,2033, \quad h(\neg e) = 0,05 \cdot 0,3515 + 0,95 \cdot 0,10925 = 0,12136.$$

Mit  $P(E)$  multiplizieren und normieren:

$$e: 0,1 \cdot 0,2033 = 0,02033, \quad \neg e: 0,9 \cdot 0,12136 = 0,10923.$$

$$\text{Summe} = 0,12956 \Rightarrow P(e \mid +j, -m) = \frac{0,02033}{0,12956} = 0,1569, \quad P(\neg e \mid \dots) = 0,8431.$$

- (i)  $P(\neg a, \neg j, m, \neg r, \neg e) = 0,03655$ .  
(ii)  $P(E \mid +j, -m) = \langle e: 0,1569, \neg e: 0,8431 \rangle$ .

### Lernziel

Teil (i) zeigt die Faktorisierung eines Bayes-Netzes. Teil (ii) ist die Kerntechnik **Variable Elimination**: statt die riesige gemeinsame Tabelle zu bilden, arbeitet man mit kleinen **Faktoren**, fasst sie zusammen (Join) und summiert versteckte Variablen heraus (sum-out). Am Ende **normieren**, weil wir nur Proportionalität berechnet haben.

## Aufgabe 12 — Bayesian Learning (7 P)

### Worum geht es?

Es gibt drei Tütentypen ( $h_1, h_2, h_3$ ) mit bekannten Anteilen (Prior) und bekanntem lime-Anteil. Wir ziehen Bonbons und **aktualisieren unseren Glauben**, welche Tüte wir haben:

$$P(h_i \mid \text{Daten}) \propto P(\text{Daten} \mid h_i) P(h_i).$$

Vorhersage des nächsten Bonbons = **Mittelung** über alle Hypothesen, gewichtet mit ihrem Posterior. Prior 0,5/0,4/0,1, lime-Anteil 0,3/0,6/1,0. Ziehungen sind i.i.d. (mit Zurücklegen).

### Lösung

(i) Posterior nach 3× lime:  $P(h_i) \cdot P(\text{lime} \mid h_i)^3$ :

$$h_1: 0,5 \cdot 0,3^3 = 0,0135, \quad h_2: 0,4 \cdot 0,6^3 = 0,0864, \quad h_3: 0,1 \cdot 1^3 = 0,1; \quad \text{Summe } 0,1999.$$

$$\Rightarrow P(h \mid 3\text{lime}) = \langle 0,0675; 0,4322; 0,5003 \rangle.$$

Vorhersage 4. Zug lime =  $\sum_i P(\text{lime} \mid h_i) P(h_i \mid \dots)$ :

$$0,3 \cdot 0,0675 + 0,6 \cdot 0,4322 + 1,0 \cdot 0,5003 = 0,7798.$$

(ii) Nach dem **1. Zug**:  $P(h_i) \cdot P(\text{lime} \mid h_i) = \langle 0,15; 0,24; 0,10 \rangle \rightarrow$  größter ist  $h_2$ . Nach dem **3. Zug** (oben): größter ist  $h_3$ .

(iii) Wahrscheinlichste Hypothese nach 3. Zug ist  $h_3$  (nur lime)  $\rightarrow P(4. \text{ Zug lime} \mid h_3) = 1$ .

(i)  $P(4. \text{ Zug lime}) = 0,7798$ . (ii) nach 1. Zug:  $h_2$ ; nach 3. Zug:  $h_3$ . (iii) 1,0 (=100%).

### Lernziel

Du lernst **Bayessches Lernen**: Beobachtungen verschieben die Wahrscheinlichkeit über Hypothesen (Prior  $\rightarrow$  Posterior). Wichtig ist der Unterschied zwischen **Bayesscher Vorhersage** (mittelt über alle Hypothesen, Teil i) und **MAP** (nimmt nur die eine wahrscheinlichste Hypothese, Teil iii).

## Aufgabe 13 — Decision Tree / Information Gain (6 P)

### Worum geht es?

Ein **Entscheidungsbaum** trennt Daten anhand von Attributen. Welches Attribut trennt am besten? Das mit dem größten **Information Gain**. Grundgröße ist die **Entropie** (Maß für Unordnung, in Bit):

$$H = -p \log_2 p - (1 - p) \log_2 (1 - p), \quad \text{Gain}(A) = H_{\text{Eltern}} - \sum_v \frac{|S_v|}{|S|} H(S_v).$$

Ziel „Reich?“: 2× Ja, 4× Nein von 6.

### Lösung

**Eltern-Entropie:**  $H = -\frac{2}{6} \log_2 \frac{2}{6} - \frac{4}{6} \log_2 \frac{4}{6} = 0,918$ .

**Attribut Größe:**

- Klein (3×): alle Nein  $\rightarrow H = 0$ .
- Groß (3×): 2 Ja, 1 Nein  $\rightarrow H = 0,918$ .

$$\text{Rest} = \frac{3}{6} \cdot 0 + \frac{3}{6} \cdot 0,918 = 0,459 \Rightarrow \text{Gain(Größe)} = 0,918 - 0,459 = 0,459.$$

**Attribut Haarfarbe:**

- Dunkel (2×): 2 Nein  $\rightarrow H = 0$ .
- Blond (2×): 1 Ja, 1 Nein  $\rightarrow H = 1$ .
- Rot (2×): 1 Ja, 1 Nein  $\rightarrow H = 1$ .

$$\text{Rest} = \frac{2}{6} \cdot 0 + \frac{2}{6} \cdot 1 + \frac{2}{6} \cdot 1 = 0,667 \Rightarrow \text{Gain(Haarfarbe)} = 0,918 - 0,667 = 0,252.$$

**(ii)** Größer ist Gain(Größe)  $\rightarrow$  auf **Größe** splitten.

Gain(Größe) = 0,459, Gain(Haarfarbe) = 0,252. **Split auf Größe.**

### Lernziel

Du verstehst **Entropie** als Maß für Unreinheit und **Information Gain** als Maß, wie stark ein Attribut die Daten „aufräumt“. Das ist die Grundregel, nach der Entscheidungsbäume (ID3/C4.5) ihre Splits wählen.

## Aufgabe 14 — Huffman-Entropie (5 BP)

### Worum geht es?

**Huffman-Kodierung** gibt häufigen Symbolen kurze Codes, seltenen lange. Die mittlere Codelänge (in Bit) nähert die Entropie an:

$$\bar{L} = \sum_i p_i \cdot \ell_i \quad (\ell_i = \text{Tiefe des Symbols im Baum}).$$

Häufigkeiten: Zocken 5, Lernen 2, TV 1, Party 1, Kneipe 1 (Summe 10).

### Lösung

**Baum bauen** (immer die zwei kleinsten Knoten verschmelzen):

$\{Z5, L2, TV1, P1, K1\} \rightarrow (TV+P) = 2 \rightarrow (K+(TV+P)) = 3 \rightarrow (L+(\dots)) = 5 \rightarrow (Z+(\dots)) = 10$

Codelängen:  $Z=1$ ,  $L=2$ ,  $K=3$ ,  $TV=4$ ,  $P=4$  (eine gültige Huffman-Struktur).

### Mittlere Länge:

$$\bar{L} = \frac{5}{10} \cdot 1 + \frac{2}{10} \cdot 2 + \frac{1}{10} \cdot 3 + \frac{1}{10} \cdot 4 + \frac{1}{10} \cdot 4 = 0,5 + 0,4 + 0,3 + 0,4 + 0,4 = 2,0.$$

(Jeder korrekte Huffman-Baum liefert  $\bar{L} = 2,0$ ; z.B. auch Längen 1, 3, 3, 3, 3.)

Huffman-Entropie  $\bar{L} = 2,0$  Bit pro Symbol.

### Lernziel

Du siehst den Zusammenhang zwischen **Kompression und Entropie**: die optimale mittlere Codelänge liegt nahe an der Shannon-Entropie ( $\approx 1,96$  Bit hier). Der gierige Huffman-Algorithmus (zwei Kleinste verschmelzen) baut diesen optimalen Präfixcode.

## Aufgabe 15 — Alldiff-Propagation / Marriage Theorem (4 BP)

### Worum geht es?

Das **Alldiff**-Constraint verlangt, dass alle Variablen verschiedene Werte bekommen. Das **Heiratssatz von Hall** sagt: Eine vollständige Zuordnung existiert nur, wenn **jede** Gruppe von  $k$  Variablen zusammen mindestens  $k$  verschiedene Werte hat. Findet man eine Gruppe von  $k$  Variablen mit **genau**  $k$  Werten (eine „Hall-Menge“), dann sind diese Werte für sie **reserviert** und müssen aus allen anderen Domains verschwinden.

Domains:  $D_A = \{2, 3, 4\}$ ,  $D_B = \{1, 2\}$ ,  $D_C = \{2, 3\}$ ,  $D_D = \{1, 2\}$ .

### Lösung

1. Betrachte  $\{B, D\}$ :  $D_B \cup D_D = \{1, 2\}$ , genau **2** Werte für **2** Variablen  $\rightarrow$  Hall-Menge. Werte 1, 2 sind für  $B, D$  reserviert  $\rightarrow$  entferne 1, 2 aus  $A$  und  $C$ :

$$D_A = \{3, 4\}, \quad D_C = \{3\}.$$

2.  $\{C\}$ : nur noch  $\{3\} \rightarrow 3$  ist für  $C$  reserviert  $\rightarrow$  entferne 3 aus  $A$ :

$$D_A = \{4\}.$$

$$D_A = \{4\}, \quad D_B = \{1, 2\}, \quad D_C = \{3\}, \quad D_D = \{1, 2\}.$$

### Lernziel

Du lernst eine stärkere Propagation als bei einzelnen Bögen: das **globale** Alldiff-Constraint erkennt „festgelegte“ Wertgruppen (Hall-Mengen) und räumt dadurch viel mehr auf. Idee: zähle Variablen vs. verfügbare Werte.

## Aufgabe 16 — Policy Iteration (7 BP)

### Worum geht es?

**Policy Iteration** ist eine zweite Methode, die optimale Strategie zu finden. Sie wechselt zwischen:

- **Policy Evaluation:** die Werte einer **festen** Strategie  $\pi$  berechnen — hier **ohne** Maximum, also ein lineares Gleichungssystem.
- **Policy Improvement:** aus diesen Werten eine bessere Strategie extrahieren (jetzt **mit** Maximum).

Start  $\pi_0(\text{Cool}) = \pi_0(\text{Warm}) = \text{Fast}$ ,  $\gamma = 0,75$ .

### Lösung

**(a) Evaluation** (Aktion fest = Fast):

$$V(\text{Warm}) = -6 + 0,75 V(\text{Warm}) \Rightarrow 0,25 V(\text{Warm}) = -6 \Rightarrow V(\text{Warm}) = -24.$$

$$\begin{aligned} V(\text{Cool}) &= 0,4(2 + 0,75V(\text{Cool})) + 0,6(2 + 0,75V(\text{Warm})) = -8,8 + 0,3 V(\text{Cool}). \\ 0,7 V(\text{Cool}) &= -8,8 \Rightarrow V(\text{Cool}) = -12,57. \end{aligned}$$

**(b) Improvement** — je Zustand beste Aktion mit diesen Werten:

$$\text{Cool: } Q_{\text{Slow}} = 1 + 0,75(-12,57) = -8,43 > Q_{\text{Fast}} = -12,57 \Rightarrow \pi_1(\text{Cool}) = \text{Slow}.$$

$$\text{Warm: } Q_{\text{Slow}} = -15,29 > Q_{\text{Fast}} = -24 \Rightarrow \pi_1(\text{Warm}) = \text{Slow}.$$

**(c) Abbruchbedingung:** Stoppe, wenn sich die Policy **nicht mehr ändert** ( $\pi_{i+1} = \pi_i$ ).

- (a)  $V^{\pi_0}(\text{Cool}) = -12,57$ ,  $V^{\pi_0}(\text{Warm}) = -24$ .
- (b)  $\pi_1(\text{Cool}) = \text{Slow}$ ,  $\pi_1(\text{Warm}) = \text{Slow}$ .
- (c) Stopp, sobald  $\pi_{i+1} = \pi_i$  (Policy stabil).

### Lernziel

Du siehst die **Alternative zu Value Iteration:** Bewerten einer festen Strategie (Gleichungssystem, kein Max!) und dann Verbessern. Beachte: bei der Evaluation ist die Aktion fixiert, beim Improvement wird über alle Aktionen maximiert. Terminierung, wenn die Strategie stabil ist.

## Aufgabe 17 — Naive Bayes (5 P)

### Worum geht es?

Ein **Naive-Bayes-Klassifikator** ordnet ein Dokument einer Klasse zu. „Naiv“ heißt: die Wörter werden als **bedingt unabhängig** gegeben die Klasse angenommen:

$$P(c \mid \text{Wörter}) \propto P(c) \prod_w P(w \mid c).$$

Mit **Laplace-Glättung** (add-1) gegen Nullwahrscheinlichkeiten:  $P(w|c) = \frac{\text{count}(w,c)+1}{N_c+|V|}$ .

Klassen: Yes (Doks 1,2), No (Doks 3,4,5). Vokabular  $|V| = 8$  (Neuer, Buer, Fraisl, Büskens, Ulreich, Bayern, München, Meister). Wörter:  $N_{\text{Yes}} = 5$ ,  $N_{\text{No}} = 6$ .

### Lösung

**Prior:**  $P(\text{Yes}) = \frac{2}{5}$ ,  $P(\text{No}) = \frac{3}{5}$ . Testdokument: „Neuer Fraisl Neuer“ (Neuer  $\times 2$ , Fraisl  $\times 1$ ).

**Benötigte Likelihoods** ( $\text{count}(\text{Neuer}, \text{Yes})=1$ ,  $\text{count}(\text{Fraisl}, \text{Yes})=2$ ,  $\text{count}(\text{Neuer}, \text{No})=2$ ,  $\text{count}(\text{Fraisl}, \text{No})=0$ ):

$$P(\text{Neuer}|Y) = \frac{2}{13}, \quad P(\text{Fraisl}|Y) = \frac{3}{13}; \quad P(\text{Neuer}|N) = \frac{3}{14}, \quad P(\text{Fraisl}|N) = \frac{1}{14}.$$

**Scores:**

$$\text{Yes} : \frac{2}{5} \left( \frac{2}{13} \right)^2 \frac{3}{13} = 0,002185, \quad \text{No} : \frac{3}{5} \left( \frac{3}{14} \right)^2 \frac{1}{14} = 0,001968.$$

$0,002185 > 0,001968 \rightarrow$  Klasse **Yes**. (Ohne Glättung wäre  $P(\text{Fraisl}|N) = 0$ ,  $\text{Score}(\text{No})=0 \rightarrow$  ebenfalls Yes.)

Klasse des Testdokuments (Schalke): **Yes**.

### Lernziel

Du lernst Textklassifikation mit Naive Bayes: Prior  $\times$  Produkt der Wort-Likelihoods, und **warum Glättung** nötig ist (ein einziges unbekanntes Wort würde sonst die ganze Klasse auf 0 setzen). Es müssen nur die tatsächlich auftretenden Wörter berechnet werden.

## Aufgabe 18 — Expectimax (5 BP)

### Worum geht es?

**Expectimax** ist wie Minimax, aber statt eines Min-Gegners gibt es teils **Zufall** (Chance-Knoten, die Kreise). Ein Chance-Knoten liefert den **Erwartungswert** seiner Kinder:

$$V(\text{Chance}) = \sum_i p_i V(\text{Kind}_i).$$

Baum: Wurzel MAX  $\rightarrow$  zwei MIN-Knoten  $\rightarrow$  je zwei Chance-Knoten  $\rightarrow$  je zwei Blätter.  
Wahrscheinlichkeiten: links  $c_1(\frac{1}{2}, \frac{1}{2})$ ,  $c_2(\frac{2}{3}, \frac{1}{3})$ ; rechts  $c_3(\frac{1}{10}, \frac{9}{10})$ ,  $c_4(\frac{3}{5}, \frac{2}{5})$ . Belohnungen (je 1×): 11, 13, 16, 18, 20, 22, 25, 28.

### Lösung

**(a) Beste Verteilung.** Die Wurzel (MAX) nimmt den **größeren** der beiden MIN-Werte; jeder MIN-Wert ist das **Minimum** seiner zwei Chance-Erwartungswerte. Also: einen Zweig mit den **vier größten** Zahlen so füllen, dass beide Chance-Werte hoch **und ausbalanciert** sind (denn das Minimum zählt).

Der **rechte** Zweig ist am stärksten, weil die  $\frac{9}{10}$ - und  $\frac{3}{5}$ -Gewichte große Zahlen stark hebeln:

$$c_3(\frac{1}{10}, \frac{9}{10}) : 22 \text{ auf } \frac{1}{10}, 25 \text{ auf } \frac{9}{10} \Rightarrow E = 0,1 \cdot 22 + 0,9 \cdot 25 = 24,7,$$

$$c_4(\frac{3}{5}, \frac{2}{5}) : 28 \text{ auf } \frac{3}{5}, 20 \text{ auf } \frac{2}{5} \Rightarrow E = 0,6 \cdot 28 + 0,4 \cdot 20 = 24,8.$$

$$\text{MIN}_{\text{rechts}} = \min(24,7, 24,8) = 24,7.$$

Die vier kleinen Zahlen  $\{11, 13, 16, 18\}$  kommen in den linken Zweig (z.B.  $c_1:16, 18 \Rightarrow 17$ ;  $c_2:13 \text{ auf } \frac{2}{3}, 11 \text{ auf } \frac{1}{3} \Rightarrow 12,33$ ;  $\text{MIN}_{\text{links}} = 12,33$ ).

### (b) Knotenwerte:

$$c_1 = 17, c_2 = 12,33, \text{MIN}_{\text{links}} = 12,33; \quad c_3 = 24,7, c_4 = 24,8, \text{MIN}_{\text{rechts}} = 24,7; \quad \textbf{Wurzel} = \max(12,33, 24,7) = 24,7$$

Optimaler Wurzelwert = 24,7. Rechter Zweig gewinnt:  $c_3(22 | 25) = 24,7$ ,  $c_4(28 | 20) = 24,8$ .  
Linker Zweig (kleine Zahlen):  $\text{MIN} = 12,33$ .

### Lernziel

Du verstehst **Erwartungswerte** in Suchbäumen und dass MAX/MIN/Chance sich mischen lassen. Der Design-Teil (a) schult die Intuition, dass **hohe Wahrscheinlichkeitsgewichte** die wertvollsten Belohnungen tragen sollten und dass ein MIN-Knoten seine **schwächere** Kante bestraft (deshalb ausbalancieren).

## Gesamtüberblick der Ergebnisse

| Aufgabe | Thema             | Kernergebnis                                                                                                 |
|---------|-------------------|--------------------------------------------------------------------------------------------------------------|
| 1       | Suche             | BFS bis $K_7$ ; DFS bis $K_{10}$ ;<br>IDS bis $K_7$ ; UCS bis<br>$K_7(15)$ ; v-viii:<br>$W/F/W/[C^*/\alpha]$ |
| 2       | Alpha-Beta        | Wurzel = 5; optimal nur 3<br>Blätter besucht                                                                 |
| 3       | CSP Rätsel        | Tee: Franzose; Hamster:<br>Belgier                                                                           |
| 4       | AC-1              | $D_1\{2, 3\}, D_2\{4\}, D_3\{3\}, D_4\{3\}, D_5\{1, 2\}$                                                     |
| 5       | Lösungsrelation   | $\{(5, 1, 5, 2), (5, 3, 5, 3)\}$                                                                             |
| 6       | Value Iteration   | $V_2(C)=2,56, V_2(W)=1,48$ ;<br>$\pi^*=(\text{Fast}, \text{Slow})$                                           |
| 7       | Q-Learning        | $\pi^*(A)=\text{Stop}, \pi^*(B)=\text{Stop}$                                                                 |
| 8       | Markov-Kette      | $P=\langle 0,4353; 0,2997; 0,2650 \rangle$ ;<br>kein Kobold: 0,3971                                          |
| 9       | HMM               | $\langle +w:0,5625, -w:0,4375 \rangle$                                                                       |
| 10      | Full Joint        | (a) $\langle 0,55; 0,45 \rangle \dots$ (d)<br>0,3333                                                         |
| 11      | Variable Elim.    | (i) 0,03655; (ii)<br>$\langle e:0,1569, \neg e:0,8431 \rangle$                                               |
| 12      | Bayesian Learning | $P(4. \text{lime})=0,7798$ ; MAP:<br>$h_2$ dann $h_3$ ; (iii) 1,0                                            |
| 13      | Decision Tree     | Gain 0,459 vs. 0,252 $\rightarrow$<br>Split auf Größe                                                        |
| 14      | Huffman           | $\bar{L} = 2,0$ Bit                                                                                          |
| 15      | Alldiff           | $D_A\{4\}, D_B\{1, 2\}, D_C\{3\}, D_D\{1, 2\}$                                                               |
| 16      | Policy Iteration  | $V^{\pi_0}=(-12,57, -24)$ ;<br>$\pi_1=(\text{Slow}, \text{Slow})$                                            |
| 17      | Naive Bayes       | Klasse: Yes                                                                                                  |
| 18      | Expectimax        | Wurzel = 24,7                                                                                                |